

电子科技大学

计算机专业类课程

课程名称：计算机网络编程

学 院：计算机学院

专 业：[REDACTED]

[REDACTED]

[REDACTED]

指导教师：向渝

日 期： 2025 年 12 月 14 日

电子科技大学

实验报告

实验一

一、实验名称：针对 ECHO 服务的 TCP 客户软件的实现

二、实验学时：4

三、实验内容和目的：

掌握客户端软件的工作原理。

掌握针对 ECHO 服务的 TCP 客户端软件的编程实现方法。

四、实验原理：

使用指定编程工具编写针对 ECHO 服务的 TCP 客户端软件的源代码。

使用指定编程工具对代码进行编译和调试，生成执行程序。

在 PC 机上执行程序，检验程序访问 ECHO 服务的功能。

五、实验步骤：

1.总体设计思路

本实验采用 C 语言基于 Socket 编程接口，实现一个面向连接的 TCP ECHO 客户端程序。

程序整体流程为：

创建套接字 → 设置服务器地址 → 建立 TCP 连接 → 循环发送数据并接收回显 → 关闭连接并退出程序。

在设计过程中，重点考虑了 TCP 通信的可靠性、错误处理机制以及客户端与服务器之间的交互逻辑。

2.创建 TCP 套接字

客户端程序首先调用 `socket()` 系统调用创建一个基于 IPv4、面向连接的 TCP 套接字。若套接字创建失败，程序通过 `perror()` 输出错误信息并立即退出，以避免后续操作在非法套接字上进行。

```
sockfd = socket(AF_INET,SOCK_STREAM,0);
if (sockfd < 0) {
    error("Failed to create socket");
}
```

3.服务器地址结构体的初始化

使用 `sockaddr_in` 结构体描述服务器的 IP 地址和端口信息。程序通过 `memset()` 对结构体进行清零，设置地址族为 `AF_INET`，并使用 `htons()` 和 `inet_pton()` 分别完成端口号和 IP 地址的格式转换，确保网络字节序正确。

```
memset(&server_addr,0,sizeof(server_addr));
server_addr.sin_family=AF_INET;
server_addr.sin_port=htons(ECHO_PORT);
if (inet_pton(AF_INET, argv[1], &server_addr.sin_addr) <= 0) {
    error("Invalid ip address");
}
printf("Connecting ECHO server %s:%d...\n", argv[1], ECHO_PORT);
```

4.建立 TCP 连接

客户端通过调用 `connect()` 函数向服务器发起连接请求。当服务器端 ECHO 服务正常运行时，`connect()` 调用成功返回，标志着 TCP 三次握手过程完成，客户端与服务器建立起可靠的通信通道。若连接失败，程序会给出明确的错误提示，便于定位服务器未启动或端口不可达等问题。

```
if (connect(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
    char error_msg[100];
    snprintf(error_msg, sizeof(error_msg), "Failed to connect the server ,please
make sure ECHO service working on port%d ", ECHO_PORT);
    error(error_msg);
}
printf("Sucessfully connect to ECHO server! \n");
```

5.循环发送数据并接收服务器回显

在成功建立连接后，程序进入主循环，实现客户端与服务器的交互功能：使用 `fgets()` 从标准输入读取用户输入；

```

while(1){
    printf("Please enter the message: ");
    memset(buffer,0,BUFFER_SIZE);
    if(fgets(buffer,BUFFER_SIZE,stdin)==NULL){
        break;
    }
}

```

对输入数据进行换行符处理，并判断是否为退出命令 quit:

```

    buffer[strcspn(buffer, "\n")] = 0;
    if(strcmp(buffer,"quit")==0){
        break;
    }
}

```

调用 send() 函数将用户输入的数据发送至服务器:

```

    if(strlen(buffer)==0){
        continue;
    }
    bytes_sent=send(sockfd,buffer,strlen(buffer),0);
    if(bytes_sent<0){
        error("Failed to send the message\n");
    }
    printf("Already sent!\n");

```

调用 recv() 函数接收服务器返回的回显数据;将接收到的数据输出到终端。

```

    memset(buffer,0,BUFFER_SIZE);
    bytes_received=recv(sockfd,buffer,BUFFER_SIZE-1,0);

```

通过该循环，可以反复验证客户端发送数据与服务器回显数据的一致性。

6.异常处理与连接关闭

在数据接收过程中，程序对 recv() 的返回值进行了判断：返回值小于 0 表示接收出错；返回值等于 0 表示服务器已关闭连接；

返回值大于 0 表示成功接收到数据。当用户输入 quit 或服务器关闭连接时，程序跳出循环，调用 close() 释放套接字资源，确保程序正常结束。

```

    bytes_received=recv(sockfd,buffer,BUFFER_SIZE-1,0);
    if(bytes_received<0){
        error("Failed to receive the data\n");
    }else if (bytes_received==0){
        printf("Server has closed the connection\n");
        break;
    }else{
        buffer[bytes_received]='\0';
        printf("Received the data: %s\n",buffer);
    }
}
close(sockfd);
printf("Connection has closed\n");
return 0;

```

六、实验数据及结果分析:

1.程序编译结果分析

从编译过程可以看到，使用 gcc 编译该 TCP 客户端程序未产生任何错误或警告，说明程序语法正确，所使用的 Socket 接口及相关系统调用符合 Linux 平台要求。

```
yar@2023080910002:~/computer_network$ gcc -o client_echo client_echo.c
yar@2023080910002:~/computer_network$
```

2.TCP 连接建立结果分析

程序运行后，客户端能够成功创建套接字并向指定 IP 和端口发起连接请求。当服务器端 ECHO 服务正常工作时，客户端输出“Successfully connect to ECHO server”提示信息，表明 TCP 三次握手过程成功完成。

```
yar@2023080910002:~/computer_network$ ./client_echo 127.0.0.1
Connecting ECHO server 127.0.0.1:7...
Sucessfully connect to ECHO server!
```

3.数据发送与接收结果分析

在客户端输入任意字符串后，程序调用 send() 函数将数据发送至服务器，随后通过 recv() 函数接收服务器返回的数据。

实验结果表明：客户端发送的数据内容与服务器回显的数据内容完全一致；数据长度和顺序未发生变化；说明 TCP 连接可靠，ECHO 服务功能正常。

```
yar@2023080910002:~/computer_network$ ./client_echo 127.0.0.1
Connecting ECHO server 127.0.0.1:7...
Sucessfully connect to ECHO server!
Please enter the message: Hello World!
Already sent!
Received the data: Hello World!
```

4.异常与退出处理分析

当服务器主动关闭连接或客户端输入 quit 命令时，程序能够正确退出循环并关闭套接字，未出现程序阻塞或异常退出的情况，说明程序对连接关闭情况进行了合理处理。

```
Please enter the message: quit
Connection has closed
```